

Annexe technique

De la donnée GeoJSON a la matrice d'adjacence

Comment, a partir des coordonnees, on modelise les points du graphe et on construit la matrice d'adjacence : metriques, calculs chiffres et choix methodologiques.

TIPE 13530 — Optimisation des tournées de collecte (Meknes)
Brahim Amsegt — CPGE MP — session 2026

Document de travail : tous les nombres chiffres sont recalculés sur les vrais points et peuvent être vérifiés à la main.

Vue d'ensemble de la chaine

L'objectif de cette annexe : detailler le maillon 'donnees -> graphe', c'est-a-dire comment un fichier de points GPS (GeoJSON) devient la matrice d'adjacence $n \times n$ sur laquelle tournent les algorithmes. La transformation d'une seule arete se resume ainsi :

Schema — la chaine de transformation d'une arete

Chaine de transformation d'UNE arete : du couple GPS au poids



Resultat : la case D[0][1] de la matrice d'adjacence (ici en secondes ; en metres pour step4).

Deux points GPS -> distance (haversine) -> poids temps (/ vitesse). Repete pour toutes les paires, cela remplit la matrice d'adjacence.

1. Le point de depart : lire le GeoJSON

Un fichier GeoJSON decrit des entites geographiques en texte. Chaque point de collecte est une entite 'Point' portant un couple de coordonnees. La fonction `load_points()` ne garde que les geometries 'Point' et en extrait les coordonnees :

```
lon, lat = geom["coordinates"][0], geom["coordinates"][1]
coords.append((lon, lat))
```

Deux choix methodologiques decisifs des cette ligne :

- L'ordre [longitude, latitude] (et NON l'inverse). C'est la convention GeoJSON, contre-intuitive car on dit 'lat/long' a l'oral. Une inversion placerait les points a des kilometres : c'est l'erreur classique du domaine.
- Le PREMIER point du fichier = le depot (indice 0). Ce n'est pas une donnee du GeoJSON mais une convention imposee : le sommet 0 a un statut particulier (depart fixe de Held-Karp, depart par default du PPV).

Resultat : une liste ordonnee de 43 couples (lon, lat). C'est l'ensemble V des sommets du graphe $G = (V, E, w)$.

2. La modelisation : du point GPS au sommet

Les 43 points GPS deviennent les 43 sommets V. On pose ensuite que le graphe est COMPLET : entre toute paire de points il existe une arete (un trajet possible), dont le poids resume tout l'itineraire reel par un seul nombre. Nombre d'aretes : $43 \times 42 / 2 = 903$.

C'est pour cela que le DESSIN ne peut pas toutes les montrer lisiblement, mais que la MATRICE D'ADJACENCE, elle, les contient toutes.

3. La metrique : la formule de haversine (et pas Pythagore)

Pour peser une arete il faut une distance entre deux points GPS. On ne peut PAS appliquer Pythagore directement sur (lon, lat) : la latitude et la longitude sont des ANGLES, pas des metres ; et 1 degre de longitude ne vaut pas la meme distance que 1 degre de latitude (les meridiens se resserrent vers les poles).

La fonction `haversine_m()` calcule la distance orthodromique (plus court arc sur la sphere terrestre) :

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$d = 2R \arcsin(\sqrt{a})$$

ou φ_1 = latitude (rad), λ = longitude (rad), $R = 6\,371\,000$ m (rayon terrestre moyen).
Le terme $\cos(\varphi_1)\cos(\varphi_2)$ est exactement ce qui corrige le resserrement des meridiens.

4. Calcul detaille sur les points 0 et 1 (verifiable)

Point	longitude	latitude
0 (depot)	-5,5748658	33,8617128
1	-5,5752357	33,8623562

Etapes du calcul (a suivre une fois, calculatrice en main) :

```
dphi = 0,00064335 deg = 1,1228e-5 rad
dlmb = -0,00036993 deg = -6,457e-6 rad
sin^2(dphi/2) = 3,152e-11
cos f1 . cos f2 . sin^2(dlmb/2) = 0,719e-11
a = 3,871e-11      sqrt(a) = 6,222e-6
d = 2 . 6 371 000 . 6,222e-6 = 79,3 m
```

C'est exactement la valeur de la case (0,1) de la matrice (l'aperçu affichait 79).

Verification croisee : le 'plan local'

A cette latitude : 1 deg lat ~ 111,0 km ; 1 deg lon ~ 111,0 x cos(33,86) ~ 92,2 km. Donc :

- déplacement Nord : 0,00064335 x 111 000 = 71,4 m
- déplacement Est : 0,00036993 x 92 200 = 34,1 m

$$\sqrt{71,4^2 + 34,1^2} = 79,1 \text{ m}$$

Les deux methodes coincident a l'echelle d'un quartier : cela prouve que haversine fait bien le travail sans projection manuelle. C'est un argument de robustesse a montrer au jury.

5. Le poids de l'arete : de la distance au temps

Choix central de la MCOT : le poids est un TEMPS, pas une distance, car deux trajets de meme longueur peuvent prendre des temps tres differents. Dans `build_matrix_hypothesis()` :

```
v_ms = AVG_SPEED_KMH / 3.6      # 30 km/h -> 8,333 m/s
d = haversine_m(...) * DETOUR_FACTOR
T[i][j] = d / v_ms                # temps = distance / vitesse
```

$$t(u, v) = \frac{d(u, v) \cdot \text{DETOUR}}{\bar{v}}$$

Sur l'arete 0->1 : $t(0,1) = 79,3 \times 1,0 / 8,333 = 9,5 \text{ s}$.

C'est la version 'hypothese forte' de la formule MCOT $t = d / v(\text{type_voie}) + \text{penalite}$, simplifiee par deux hypotheses assumees :

- `AVG_SPEED_KMH = 30` : une vitesse moyenne CONSTANTE (on neglige les types de voie). C'est l'hypothese forte a defendre.
- `DETOUR_FACTOR = 1,0` : distance a vol d'oiseau PURE. Le passer a ~1,3 relacherait l'hypothese en approchant les detours routiers (un trajet reel est ~30 % plus long que

la ligne droite). C'est le bouton de realisme.

6. Construire la matrice n x n complete

On remplit un tableau carre D par une double boucle sur toutes les paires :

```
for i in range(n):
    for j in range(n):
        if i != j:
            d = haversine_m(...) * DETOUR_FACTOR
            T[i][j] = d / v_ms
```

(En step4, distance_matrix() fait pareil mais s'arrete en metres et exploite la symetrie : seul le triangle superieur est calcule puis recopie.)

Trois proprietes mathematiques garanties par cette construction :

Propriete	Pourquoi	Consequence
Diagonale nulle $D[i][i]=0$	point vers lui-meme	le if $i!=j$ la laisse a 0
Symetrie $D[i][j]=D[j][i]$	haversine symetrique	graphe non oriente
Inegalite triangulaire	vraie distance metrique	TSP metrique

$$D[i][k] \leq D[i][j] + D[j][k]$$

Cette derniere propriete (inegalite triangulaire) fait de l'instance un TSP METRIQUE : elle valide le 2-opt et donne au plus proche voisin sa garantie de qualite theorique.

7. La symetrisation explicite

En mode osm, la matrice REELLE est asymetrique (sens uniques : $t(u,v) \neq t(v,u)$). Pour que le 2-opt reste valide (inverser un segment ne doit pas changer son cout), symmetrize() moyenne les deux sens :

$$D[i][j] \leftarrow \frac{D[i][j] + D[j][i]}{2}$$

En mode hypothese la matrice est DEJA symetrique : cette operation ne change rien, mais elle est appliquee par coherence. Elle reste une hypothese de modelisation a assumer, pas un detail technique.

8. Deux unites, un seul graphe

La matrice en SECONDES (step1) et la matrice en METRES (step4) decrivent le MEME graphe a un facteur d'echelle pres :

$$t = d / 8,333$$

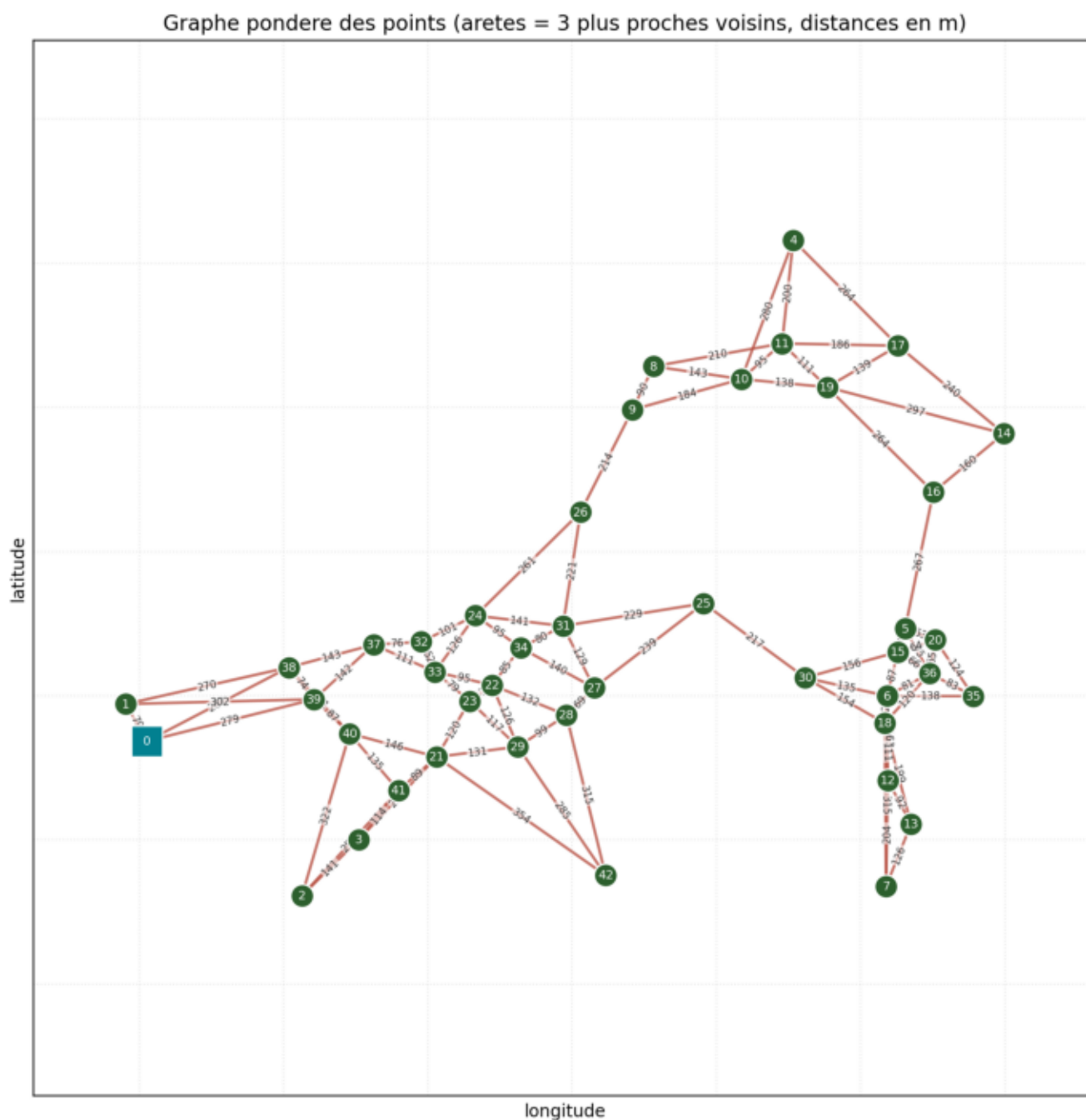
C'est pourquoi le classement des algorithmes est identique dans les deux unités : changer d'unité ne change pas quel tour est le plus court.

9. Synthèse des choix méthodologiques (à défendre)

- Convention [lon, lat] + depot = point 0 : structuration des données.
- Graphe complet : chaque trajet réel résumé par un poids unique (903 arêtes).
- Haversine plutôt qu'eucclidien plat : respecte la courbure et le resserrement des méridiens ; vérifie équivalent au plan local au quartier.
- Poids = temps, pas distance : fidélité au problème de collecte ($t = d / v$).
- AVG_SPEED = 30 km/h constante : hypothèse forte (ni type de voie, ni trafic) cohérente avec la collecte 5h-7h à Meknes.
- DETOUR_FACTOR = 1,0 : borne 'vol d'oiseau pur', réglable pour relâcher l'hypothèse.
- Symétrisation : rend le graphe non orienté pour légitimer le 2-opt.
- Deux unités, un seul graphe : mètres (lisibilité) et secondes (résolution), reliées par un facteur d'échelle.

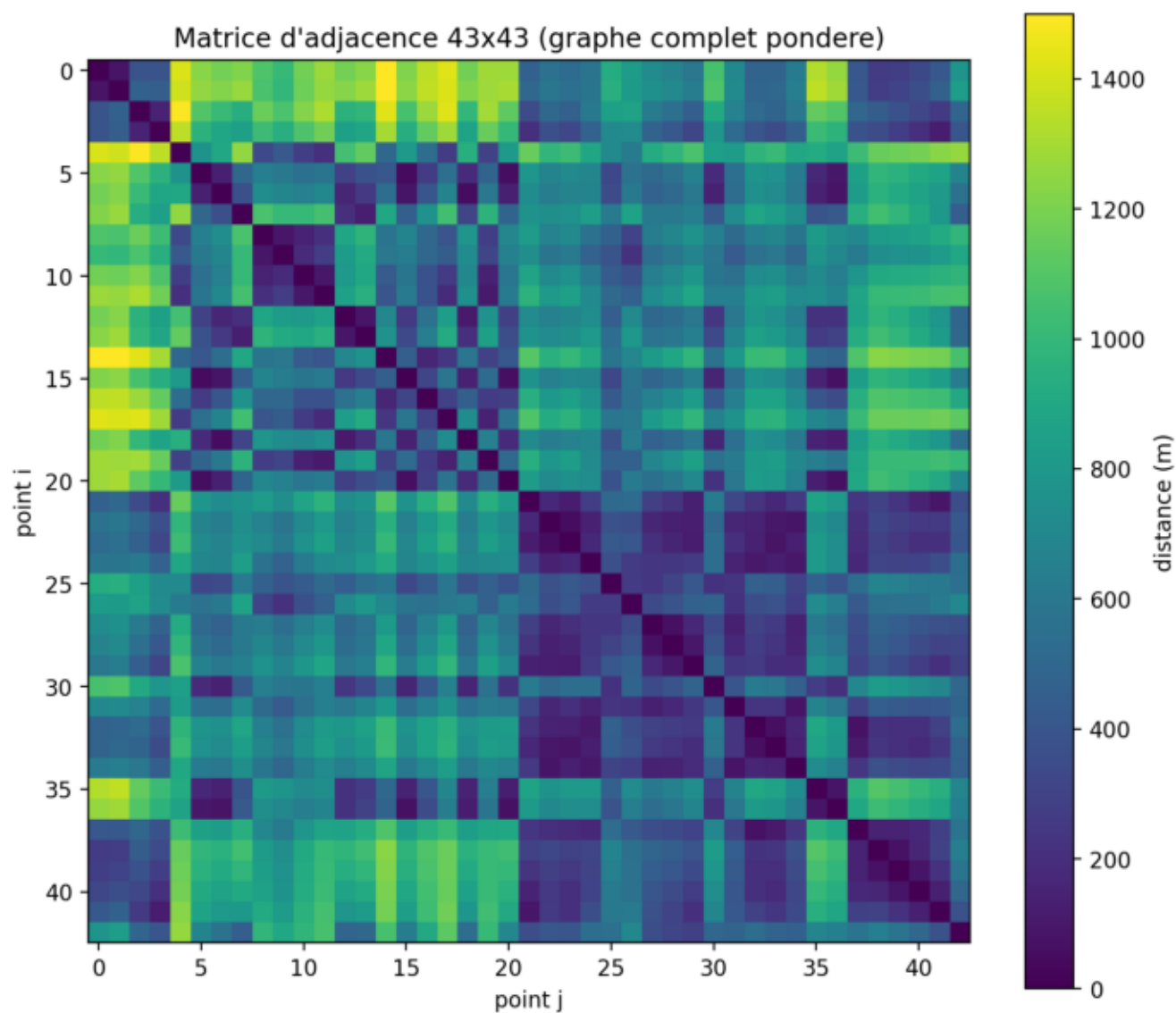
Le tout produit une matrice 43 x 43, symétrique, à diagonale nulle, métrique : l'objet mathématique propre sur lequel tournent les trois algorithmes.

Illustration — le graphe pondere obtenu



Chaque sommet a sa position GPS réelle ; chaque arete porte la distance (issue de haversine). Le carre est le depot (point 0).

Illustration — la matrice d'adjacence complete (43 x 43)



La meme information sous forme de tableau de couleurs : sombre = proche, clair = lointain ; diagonale nulle, matrice symetrique.